Understanding the HDC Classifier — From Zero

This guide assumes you know nothing about the topic. It starts from the very basics (what a bit is, what a classifier is, what EMG is) and builds up, step by step, to a complete understanding of the Hyperdimensional Computing (HDC) hand- gesture classifier built in this project. Read it top to bottom; every term is explained before it is used.

PART I — FOUNDATIONS (assume nothing)

1. What are we trying to build, and why?

We are building a small digital system that can **recognise hand gestures from muscle signals**. You place a few sensors on a person's forearm, they make a gesture (close the hand, open it, pinch two fingers, point, or rest), and the system outputs **which** of those gestures it is.

Why this matters:

- **Prosthetics:** an amputee's residual forearm muscles still fire; reading them lets a robotic hand move the right way.
- **Wearables / human-computer interaction:** silent, hands-free control.
- **It is a hard, real problem** that is a great showcase for a new, ultra- efficient style of computing (HDC) running on a small chip.

> **Plain-English goal:** muscle signal in → gesture name out, using a method so simple (just AND/XOR/counting) that it fits in a tiny, low-power chip.

2. What is EMG (the input signal)?

EMG = Electromyography. When a muscle contracts, the nerve cells fire tiny electrical pulses. Electrodes (small metal pads) on the skin pick up that electrical activity as a wiggly voltage over time.

In this project there are **4 electrodes** = **4 channels**, placed around the forearm. So at every instant we have **4 numbers** (how strongly each spot is active). Stronger muscle activity = bigger number.

The raw wiggle is messy, so it is **smoothed** (an "envelope", like tracing the outline of the wiggles) into a clean value that rises and falls with effort. In this project each channel's smoothed value lands in the range **0 to ~20**.

> **Analogy:** think of 4 volume knobs (one per electrode). A gesture is a particular *pattern* of the 4 knobs over a short time. Different gestures = different patterns.

3. What is a "classifier"?

A **classifier** is anything that takes some input and chooses one **label** (category) from a fixed list. Here the labels are the 5 gestures.

Every classifier works in two phases:

1. **Training (learning):** you show it labelled examples ("this pattern = pinch", "that pattern = rest") and it builds an internal model.
2. **Inference (using it):** you show it a *new* pattern and it outputs its best guess of the label.

> **Analogy:** training is studying flashcards; inference is the exam. We will see that HDC's "studying" is unusually fast — basically just averaging.

4. The tiny bit of math we need

You do **not** need advanced math. Just these five simple ideas.

4.1 Bits and binary

A **bit** is a single digit that is either **0** or **1**. That's it. Computers store and process everything as bits. A row of bits like `1 0 1 1` is called a **binary string** or **bit vector**.

4.2 A "vector" is just a list

A **vector** here means *an ordered list of numbers*. `[3, 0, 7, 2]` is a vector of 4 numbers. `1 0 1 1 0 0 1 0` is a vector of 8 bits. The **dimension** is how many entries it has. This project's vectors have dimension **1024** (1024 bits) — hence "**hyper**dimensional" (very high dimension).

4.3 XOR — the "are they different?" operation

XOR ("exclusive or", symbol ⊕) compares two bits and outputs `1` if they **differ**, `0` if they are the **same**:

```
a   b   a⊕b
0   0    0
0   1    1
1   0    1
1   1    0
```

For two bit vectors, XOR is done position by position:

```
1 0 1 1 0 0 1 0
0 0 1 0 1 1 1 0   XOR
---------------
1 0 0 1 1 1 0 0
```

4.4 Majority vote

Given several bits, the **majority** is the value (0 or 1) that appears most often. For three bits `1,1,0` → majority is `1`. We apply it column-by-column to several vectors. If there's a tie (equal 0s and 1s), we need a rule — this project breaks ties to **0**.

4.5 Hamming distance — "how different are two bit vectors?"

The **Hamming distance** between two bit vectors is simply **the number of positions where they differ**. You can compute it as: XOR them, then **count the 1s** (counting 1s is called **popcount**).

```
A   = 1 0 1 1 0 0 1 0
B   = 1 0 0 1 1 0 1 1
A⊕B = 0 0 1 0 1 0 0 1   -> popcount = 3   -> Hamming distance = 3
```

Small distance = **similar**. Large distance = **different**. This is the only "similarity ruler" we use.

> **You now know everything needed:** bit, vector, dimension, XOR, majority, Hamming distance. The rest is just clever ways to combine these.

PART II — THE CORE IDEA OF HDC

5. Represent *meaning* as long random bit vectors

Hyperdimensional Computing's one big idea:

> **Give every "thing" in your problem its own long random bit vector (1024 bits). Then combine and compare those vectors to compute.**

"Things" here include: each electrode/channel, each signal level, each time position, and ultimately each whole gesture. Each gets a 1024-bit vector called a **hypervector**.

A **random** hypervector has about half 0s and half 1s, chosen by (conceptual) coin flips.

6. Why make them so long? (the "magic" of high dimension)

Here is the crucial insight. Take **two random 1024-bit vectors** and measure their Hamming distance. Because each bit is an independent coin flip, on average they differ in **half** their bits:

```
average distance between two random vectors = 1024 / 2 = 512 bits (~50%)
typical wobble around that average          = about ±16 bits
```

So two *unrelated* random hypervectors are almost always close to 50% apart, and it is **wildly unlikely** for two unrelated ones to be, say, only 20% apart by accident. This property is called **quasi-orthogonality** ("nearly independent").

Why this is the whole game:

- Two **different** concepts get random vectors → they sit ~50% apart → they **never get confused**.
- Two **related** concepts can be given **deliberately similar** vectors → small distance → similarity becomes *meaningful and measurable*.

- You can **pile many vectors together** and still recover information, because random noise averages out over 1024 dimensions.

> **Analogy:** in a huge stadium crowd, two random people stand far apart. You'd never mistake one for the other. But friends can choose to stand close. "Long vectors" = "huge stadium" = lots of room to keep things distinct.

The **bigger** the dimension, the more reliable this separation. (A research finding of this project: 1024 bits is already plenty for the spatial task — see Part VI.)

PART III — THE THREE OPERATIONS (the toolkit)

HDC computes using just **three** operations plus the similarity ruler. We show each with a tiny **8-bit** example so you can follow every bit. The real system uses 1024 bits, but the rules are identical.

7. BIND = XOR ("tie two things together")

Bind combines two hypervectors into a new one. We use XOR.

```
A    = 1 0 1 1 0 0 1 0      (say, "channel 2")
B    = 0 0 1 0 1 1 1 0      (say, "value 7")
A⊕B  = 1 0 0 1 1 1 0 0       ("channel 2 holds value 7")
```

What bind gives you:

- The result is **unlike either input** (about 50% away from both) → it's a fresh, distinct symbol meaning "*A paired with B*".
- It is **reversible**: `(A⊕B) ⊕ B = A`. You can undo the pairing if you know one half.
- It **preserves distances**: binding two things with the *same* key keeps their relationship intact.

> **Analogy:** bind is like **stapling a name tag to a value**: "this number belongs to channel 2". The stapled bundle looks different from both the name and the number, but it precisely records the pairing.

We use bind to attach a **measured value** to the **channel it came from**.

8. BUNDLE = majority ("mix several things into one")

Bundle merges several hypervectors into a single one that is **similar to all of them** — like a set, or an average. We use column-wise **majority vote**.

```
V1  = 1 0 1 1 0 0 1 0
V2  = 1 1 1 0 0 1 1 0
V3  = 0 0 1 0 1 1 1 1
```

```
         ---------------    count the 1s in each column:
count = 2 1 3 1 1 2 3 1     (out of 3)
maj   = 1 0 1 0 0 1 1 0     (1 if count is more than half)
```

What bundle gives you:

- The result is **close to every input** → it "remembers" all of them at once.
- **Limited capacity:** mix too many and they blur (like overlapping too many transparencies). How precisely we track the vote (the **counter width**) is a hardware knob.
- Needs a **tie rule** when an even number of inputs splits 50/50 → here, **0**.

> **Analogy:** bundle is like **overlaying transparent sheets**: the stack shows what most sheets agree on. Each individual sheet is still "mostly visible" in the result.

We use bundle twice: to fuse the 4 channels into one snapshot, and to average all training examples of a gesture into one "prototype".

9. PERMUTE = cyclic shift ("stamp the time/position")

Permute (symbol `ρ`) rotates a hypervector's bits by one place. It makes a new vector that is unrelated to the original, used to mark **position in a sequence**.

```
R      = 1 0 1 1 0 0 1 0
ρ(R)   = 0 1 0 1 1 0 0 1     (everything shifted right by 1, last wraps around)
ρ²(R)  = 1 0 1 0 1 1 0 0     (shifted by 2)
```

What permute gives you:

- `ρ(R)` is ~50% away from `R`, so "R at time 1" is distinguishable from "R at time 0".
- It is **reversible** and **distance-preserving** (just a fixed reordering).

> **Analogy:** permute is like **rotating a dial one click per second**. The same reading at different times ends up at different dial positions, so order is remembered.

We use permute to record the **order** of snapshots within a gesture window.

10. SIMILARITY = Hamming distance (popcount)

To decide which stored vector a new one matches, we compute Hamming distance: `popcount(A ⊕ B)`. **Smallest distance wins.** `popcount` (counting 1s) is one cheap hardware step.

PART IV — MEMORIES (turning real data into vectors)

We have raw numbers (channel IDs 1–4, values 0–20). We need a consistent way to turn each into a hypervector. We use two lookup tables ("memories").

11. Item Memory (iM): a codebook of random vectors

For things with **no natural order** — the 4 channels — we simply assign each one a **fixed random** 1024-bit vector once, and reuse it. Because random vectors are quasi-orthogonal, "channel 1" and "channel 3" are ~50% apart and never confused.

```
iM[1] = (random 1024-bit vector)
iM[2] = (another random one)
iM[3] = ...
iM[4] = ...
```

12. Continuous Item Memory (CiM): levels with graded similarity

Sensor **values** *do* have an order: level 5 is closer to 6 than to 20. We want **similar values → similar vectors**, so that small measurement changes cause small vector changes (robustness).

We build the level vectors by starting from a random vector for level 0 and **flipping a fixed block of bits at each step up**:

Continuous item memory: each level flips the next block of bits

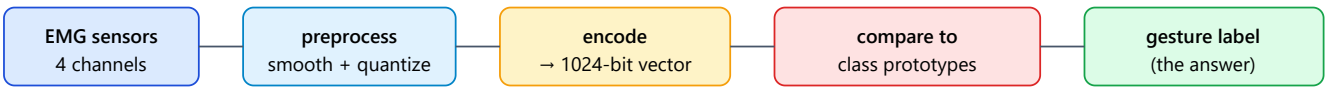
```
level 0:    0 0 0 0 0 0       | | |   (random start)
level 1:    1 1  0 0 0 0      | | |   flip first block
level 2:    1 1 1 1   0 0     | | |   flip next block
level 3:    1 1 1 1 1 1       | | |   ...and so on
```

Nearby levels share almost all bits (very similar); far-apart levels differ a lot.

Result: `distance(level a, level b)` grows smoothly with `|a − b|`. Adjacent levels are nearly identical; the extremes (0 vs 20) are ~50% apart. That's exactly the graded similarity we want from an analog amplitude.

PART V — THE CLASSIFIER, STEP BY STEP

Now we assemble the pieces. The whole system looks like this:



Step 0 — Preprocess

The smoothed EMG is **quantized**: each channel's value is rounded to a whole level **0–20**. So one instant in time = 4 small integers, e.g. `(ch1=3, ch2=0, ch3=11, ch4=7)`.

Step 1 — Encode one instant → a "record"

For each channel, **bind** its identity with its value's level vector, then **bundle** the four channels together:

```
for each channel c:   bound_c = iM[c] XOR CiM(value_c)
record R = majority( bound_1, bound_2, bound_3, bound_4 )
```

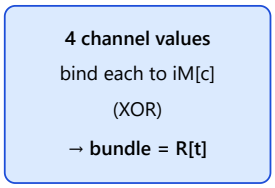
`R` is one 1024-bit vector meaning "*the 4 muscles had these 4 activity levels at this instant*".

Step 2 — Encode a short window → an "n-gram" (adds time)

A gesture is movement, not a freeze-frame. Take `N` consecutive records, mark each with its time position using **permute**, and **bundle**:

```
G = majority over t = 0..N-1 of  ρ^t ( R[t] )
```

Because `ρ^0, ρ^1, ρ^2, …` are quasi-orthogonal, the **order** is preserved ("rising then falling" ≠ "falling then rising"). `G` is the final **query hypervector** describing the whole window. (Best `N` is 3–5 here, per person.)



stamp time: $\rho^0 R[0]$, $\rho^1 R[1]$, $\rho^2 R[2]$… bundle them

query G 1024-bit window vector

Step 3 — Training: make one "prototype" per gesture

To learn a gesture, take **all its training windows**, encode each to a `G`, and **bundle them all** into a single vector:

```
prototype[gesture k] = majority over all training G of class k
```

Each prototype is a 1024-bit vector that is similar to every example of that gesture and different from the others. The collection of prototypes (one per gesture) is the **Associative Memory (AM)**.

> **That's the entire "learning".** No gradient descent, no epochs — just averaging by majority vote. Add

> more data later by bundling it in. This is why HDC can learn **on-device** from very few examples.

Step 4 — Inference: pick the nearest prototype

For a new window, encode it to `G`, then compute the Hamming distance from `G` to each prototype and choose the **closest**:

```
predicted = the gesture k whose prototype is nearest:
            argmin over k of  popcount( G XOR prototype[k] )
```

Optionally a **pruning mask** (a fixed 1024-bit on/off pattern) is ANDed in first so only the most useful bit positions are compared — smaller, faster hardware with little accuracy loss (a research knob, Part VI).

Step 5 — A FULL end-to-end example (tiny, D = 8)

Let's run the whole pipeline once with 2 channels and 8-bit vectors so every step is visible.

```
MEMORIES
  iM[1]  = 1 0 1 1 0 0 1 0          (channel 1 identity)
  iM[2]  = 0 1 1 0 1 0 0 1          (channel 2 identity)
  CiM(0) = 0 0 0 0 1 1 1 1          (value level 0)
  CiM(1) = 1 0 0 0 1 1 1 0          (value level 1; 2 bits flipped vs level 0)

ENCODE an instant where channel1 = level 1, channel2 = level 0
  bound_1 = iM[1] XOR CiM(1) = 1 0 1 1 0 0 1 0 ⊕ 1 0 0 0 1 1 1 0 = 0 0 1 1 1 1 0 0
  bound_2 = iM[2] XOR CiM(0) = 0 1 1 0 1 0 0 1 ⊕ 0 0 0 0 1 1 1 1 = 0 1 1 0 0 1 1 0
  column 1-counts of (bound_1,bound_2) = 0 1 2 1 1 2 1 0   (out of 2)
  record R = majority, tie->0 (need >1) =  0 0 1 0 0 1 0 0   ← this is our query G
(N=1)

PROTOTYPES (suppose training produced)
  P[A] = 0 0 1 0 0 1 0 1
  P[B] = 1 1 0 0 1 0 1 0

CLASSIFY G = 0 0 1 0 0 1 0 0
  G XOR P[A] = 0 0 0 0 0 0 0 1  -> popcount = 1   (distance to A = 1)
  G XOR P[B] = 1 1 1 0 1 1 1 0  -> popcount = 6   (distance to B = 6)
  nearest = A   →   PREDICTED GESTURE = A
```

The real system does exactly this, only with 1024-bit vectors, 4 channels, 21 levels, 5 gestures, and several time steps per window.

PART VI — WHY HARDWARE LOVES THIS, AND THIS PROJECT'S NUMBERS

13. Why HDC is perfect for an FPGA / Zynq chip

Look back at every operation: XOR, majority (counting), shifting bits, popcount, AND. **None** of them need multiplication or decimals. They are the cheapest things a chip can do, and all 1024 bits can be processed **in parallel** in a handful of clock cycles. A neural network needs lots of multiply-accumulate hardware and number formats; HDC needs gates and counters. That is the core hardware advantage.

> **Zynq** is a chip with two parts: a small CPU ("PS", the ARM processor) and a reconfigurable logic fabric ("PL", the FPGA). We build the HDC engine in the fast PL fabric, and the CPU just feeds it data and reads the answer.

14. How the math maps onto the actual hardware modules

Pipeline step	Operation	Hardware building block	RTL file
Look up channel/level vectors	table read	ROM / BRAM	`item_memory.sv`
Bind	XOR	XOR gates	`xor_permute_top.sv`
Permute	cyclic shift	wiring / rotate	`permute_stage.sv`
Bundle	majority vote	per-bit counters + threshold	`bundle_unit.sv`
Pruning	per-bit AND	mask register	`pruning_mask.sv`
Similarity + decision	XOR + popcount + smallest	popcount tree + comparator	`popcount_am.sv`
Talk to the CPU	registers / streaming	AXI4-Lite / AXI-Stream	`hdc_axi_lite_wrapper.sv`, `hdc_stream_wrapper.sv`

15. This project's exact settings

- **Dimension D = 1024 bits** (headline; the design is parameterised so D can be changed).
- **4 channels, 5 gestures**, EMG envelope quantized to **21 levels (0–20)**.
- **Bind = XOR, permute = cyclic shift, bundle = majority (tie→0), similarity = Hamming/popcount.**
- **Training = majority-bundled prototypes** (fast, few-shot).

16. The measured results (does it actually work?)

We reproduced a well-known published benchmark (Rahimi et al., 2016) on its own data, and then ran our binary 1024-bit version on the same data:

Model	Spatial accuracy	Spatiotemporal accuracy
This project (binary, D=1024)	**90.30 % ± 0.13**	89.25 % ± 0.89
Bipolar reference (D=10000)	90.36 %	96.04 %
Original paper	90.8 %	97.8 %

The headline: our small binary 1024-bit design matches the much larger 10000-dimensional reference on the spatial task. That's strong evidence the hardware-friendly version is sound.

17. The "knobs" the research turns (where the novelty is)

HDC lets us trade a little accuracy for big savings in chip area and energy. The project systematically studies:

- **D (dimension):** fewer bits → smaller/faster/lower-power. We found spatial accuracy is nearly flat from D=256 up to 10000, so there's lots of room to shrink.
- **Counter width (CNT_W):** how precisely the bundle's majority counters work; lower precision saves flip-flops.
- **Bit pruning:** keep only the most *useful* bit positions. "**Informed**" pruning (keep the discriminative bits) vs "**random**" pruning — showing informed beats random is a key claim.
- **Cross-subject masks:** can a pruning pattern learned on one person work on another? (A generalisation study.)

PART VII — REFERENCE

18. Frequently asked questions

Q: Isn't a random vector useless? How can randomness carry meaning? The meaning isn't in any single vector — it's in the *relationships*. Random vectors are guaranteed to be distinct (quasi-orthogonal), so they make perfect, non-colliding "name tags". Meaning emerges when we bind/bundle them.

Q: Why 1024 and not 10 or a million? Too few bits and unrelated things collide (errors). Too many and you waste chip area for no accuracy gain. 1024 is a sweet spot we verified experimentally.

Q: How is training so fast with no neural network? A class is just the *average* (majority bundle) of its examples. Averaging is one pass over the data — no iterative optimisation. The trade-off is slightly lower peak accuracy than a big trained network, in exchange for tiny size, instant learning, and robustness.

Q: What makes it robust? Information is spread across all 1024 bits. Flipping a few (noise, a dead bit, a pruned position) changes the Hamming distance only slightly, so the nearest prototype is usually still correct.

Q: Why binary instead of the ±1 "bipolar" version in the original paper? Binary (`0/1`) maps directly to XOR + popcount, which is what cheap hardware does best. We showed binary at D=1024 matches bipolar at D=10000 for the spatial task.

19. Glossary

Term	Plain meaning
Bit	A single 0 or 1.
Vector	An ordered list of numbers/bits.
Dimension (D)	How many entries a vector has (here 1024).
Hypervector	A very long (1024-bit) vector representing one concept.
HDC / VSA	Hyperdimensional Computing / Vector Symbolic Architecture.
BSC	Binary Spatter Code — the 0/1 HDC style used here.
EMG	Electromyography — electrical signal from muscles.
Channel	One electrode's signal (4 of them here).
Quantize	Round a value to a whole level (0–20).
XOR (⊕)	Bit op: 1 if two bits differ, else 0.
Bind	Combine two vectors (XOR) into a new, reversible pairing.
Bundle	Merge vectors (majority) into one similar to all — a set/average.
Permute (ρ)	Cyclic bit-shift; stamps order/position.
popcount	Count the 1 bits.
Hamming distance	Number of differing bits = popcount(A⊕B); the similarity ruler.
Quasi-orthogonal	Random hi-D vectors sit ~50% apart, so they act independent.
Item Memory (iM)	Codebook of random vectors for unordered IDs (channels).
Continuous IM (CiM)	Level vectors with graded similarity for ordered values.
Record (R)	One-instant vector fusing the 4 channels.
n-gram (G)	Permute+bundle of N records → the window/gesture vector (query).
Prototype	A gesture's representative vector (bundle of its examples).
Associative Memory (AM)	The set of all class prototypes.
Pruning mask	Fixed on/off pattern selecting which bit positions to use.
FPGA / PL	Reconfigurable logic fabric where the HDC engine runs.
Zynq / PS	The chip; PS is its ARM CPU that feeds the engine.
AXI	The bus protocol the CPU uses to talk to the engine.

20. Where to read more

- P. Kanerva, "*Hyperdimensional Computing: An Introduction to Computing in Distributed Representation with High-Dimensional Random Vectors,*" 2009 — the gentle foundational tutorial.
- A. Rahimi, S. Benatti, P. Kanerva, L. Benini, J. Rabaey, "*Hyperdimensional Biosignal Processing: A Case Study for EMG-based Hand Gesture Recognition,*" IEEE ICRC 2016 — the anchor paper this project reproduces and extends.

You now have the full picture: muscle signal → 4 quantized values → bind each to its channel and bundle → permute+bundle across time into one 1024-bit query → compare (Hamming) to averaged class prototypes → nearest one is the predicted gesture, all with cheap bitwise hardware.